

AXIS 0

Programs made of fields

Packed Boolean definitions, expressions and temporal self-reference

Attribution	Supplied identity
Name	Tom Klootwijk
Identifier	NL200678942
Date as supplied	10-07-1990

The central change

This is a **field-program calculator**, not a rendering engine and not merely a fixed routing rule. Operator definitions, expression wiring and state use packed field representations. A calculation can quote an operator's own definition, produce a new definition, and use that definition at the next **A = T** snapshot.

The package includes a working CPU reference, CUDA source for the RTX 5070 Ti Laptop target, an exact SDF coding construction, examples, exhaustive finite checks, and scripts to measure the actual laptop.

Executed here: CPU compilation, correctness tests, sanitizer checks and CPU comparisons.

Not executed here: CUDA compilation or GPU benchmarks; this environment has neither nvcc nor an NVIDIA GPU.

18 September 2026 • ELI5 guide with technical contracts and measured results

01 / SOURCE BASIS AND SELECTED REALIZATION

Your goal, made explicit

The original definition says Axis 0 is the domain of **ALL admitted operators**, with the operators themselves defined as SDFs. The new Boolean double UU instruction asks for both the field and the expressed operator to be Boolean fields. The current request adds that **all expressions** must be field objects and that **A is T**. [S1, p. 5; S3, pp. 2–3; current request]

ELI5: recipes made of the same material as ingredients

Imagine a calculator whose ingredients are strips of tiny on/off lamps. Its recipes are also strips of lamps. It can read a recipe, calculate with it, inspect that recipe's own lamps, and prepare a changed recipe for the next round. It still needs a small, fixed mechanism that knows how to read the lamps.

The specific interpretation used here

Every admitted primitive operator is an eight-bit truth field. Every expression is a packed field that connects other fields. Every state value is one bit per expression per independent case. More complicated Boolean expressions and bounded integers are built by composing these fields. The physical GPU remains the machine that executes the reading rule.

What the Shannon link supports

The linked educational document explains Boolean circuit synthesis and reconstructing functions from their truth coefficients. That supports the use of a Boolean evaluator. I did not locate the exact “raw binary logic” sentence in its accessible text. The sentence is treated as **your implementation goal**, not as a measured GPU-speed or cache-residency theorem. [E1; E2]

The goal is achieved narrowly and concretely: **the calculation path does not evaluate floating-point coordinates, logarithms, trigonometry or distances**. It reads pre-encoded bits and uses Boolean selections. This does not prove that arbitrary geometry or numerical information can be removed without changing a problem.

The source files also contain accompanying generated explanations. Their claims about instant TSP solutions, thermodynamic halting or universal self-replication are not used as executable facts. New implementation choices are identified throughout this guide.

02 / OPERATOR FIELDS, EXPRESSION FIELDS AND STATE FIELDS

What is actually stored

Object	Packed contents	What the evaluator reads
Operator field	Eight truth bits	The output for each of the eight possible three-input assignments.
Expression field	256 bits / 32 bytes	An operator-field address, three references, time-delay flags, row label, identity and seed convention.
State field	One bit per case	The current Boolean result of one expression in each independent calculation.
Publication field	128 bits / 16 bytes	Which calculated bit changes which custom operator coefficient at the next tick.
Quoted field	An ordinary bit reference	A bit of an operator or expression, read as a calculation input.

All operators of a declared finite basis

There are exactly **256 possible Boolean functions of three input bits**: eight possible inputs, each with an independently chosen zero/one output. The complete catalogue occupies **256 bytes**. AND, OR, XOR, NOT, NAND, majority and selection are aliases for particular truth fields. No C++ opcode switch supplies their individual meanings.

```
input index = a + 2*b + 4*c
result bit = truth_field[input index]
```

Thirty-two assignments are evaluated together in one unsigned 32-bit word. Larger functions are networks of these operator fields. This is “all admitted operators” for the implemented Boolean language, not a claim that every opaque phrase in the original documents already has an executable definition.

The evaluator can itself be described as fields

The included **interpreter_in_fields** example expresses the Boolean selection rule as seven MUX operator fields. It was checked on every one of its **2,048** argument/coefficient combinations. The underlying GPU reader remains a fixed implementation substrate.

In what sense are these SDFs?

This revision provides an explicit geometric definition rather than simply relabeling a byte as an SDF. For a finite bitword b , put a unit-radius interval around coordinate $4j$ whenever bit j is one. Always include a sentinel interval centred at -4 . The intervals are disjoint.

```
Omega_b = [-5,-3] union [4*j-1,4*j+1] for each b[j]=1
D_b(q) = min over included centres c of (abs(q-c)-1)
b[j] = 1 exactly when D_b(4*j) < 0
```

Because the intervals are disjoint, this is an exact signed Euclidean distance to their union boundary. Inside an interval the value is negative; on its boundary it is zero; outside all intervals it is positive. The sentinel ensures the all-zero word also has a defined boundary.

Initial LIVE coefficient j	0	1	2	3	4	5	6	7
Stored bit	0	1	0	1	0	1	0	1
D at anchor 4j	3	-1	3	-1	3	-1	3	-1

The optimization is a proved reading identity

The running program needs only the negative-distance test at a known anchor. That test is **exactly the stored bit**, so the implementation can read it directly. It never needs to calculate the coordinate or distance in the execution loop. The same construction applies to operators, expression records, published rules and result words.

Two spaces must not be confused

The **coding line** above gives every finite bitword an SDF interpretation. The **log-polar storage chart** tells the GPU where the packed word is stored. They are distinct spaces. This construction is introduced for this prototype; it is not a claimed implementation of the source's Klein-bottle geometry.

The codec does not losslessly replace arbitrary original distance samples with one bit. It defines a particular geometric object from the bits. The exact codec was tested on 256 eight-bit words and 12,800 integer queries. [Local verification: integration_tests.json]

04 / TIME IS A DEFINED DEPENDENCY, NOT AN UNSTATED CLOCK

A = T has a concrete rule

Here **A** and **T** name the same previous-snapshot operator. This is an explicit implementation choice for the temporal language in the request. It is not a deduction about physical time from the shape of a field. [S2, pp. 7–8; current request]

$T(x)[n+1] = x[n]$
 $A(x)$ is an alias for $T(x)$

Current references and delayed references

A current reference reads an expression already calculated in this tick. A delayed reference reads the previous snapshot and may point back to the same expression. That gives a definite meaning to recursive state equations. An unclocked cycle is rejected rather than assigned a guessed fixed point.

Expression idea	Effect
ID(T(x))	Keep the previous value of x.
NOT(T(x))	Toggle x once per tick.
Four clocked bit fields plus carry expressions	Increment a four-bit counter without floating-point arithmetic.

A measured example

The counter example starts case i with the four-bit value $i \bmod 16$. After n ticks, the decoded value is $(i + n) \bmod 16$. It passed checks at $n = 0, 1, 2, 7, 15, 16, 17$ and 63 for every supplied case.

ELI5: yesterday’s page and today’s page

The calculator has two pages of bits. **T** tells an expression to read yesterday’s page. Other references may read results already written on today’s page. At the end of the tick the pages exchange roles. Older history is not retained unless separately recorded.

The alpha component **A** is therefore a temporal snapshot label, not a graphics opacity value. The source’s **SDF = T** terminology is retained as the requested interpretation, while the actual finite clock law is stated above. No thermodynamic halting rule or continuous metric-time dynamics is inserted.

05 / REFLECTION AND PUBLICATION ARE IMPLEMENTED

A definition that changes itself

The **self_definition** example goes beyond feeding ordinary results back as inputs. It reads the eight coefficient bits of its own custom operator field LIVE, calculates their complements, and publishes those bits into LIVE's next snapshot.

```
LIVE starts as ID: 0xaa
quote LIVE -> complement its coefficients -> publish to LIVE
next tick uses NOT: 0x55
repeat -> ID -> NOT -> ID ...
```

Tick	Definition used	Output for input 0,1,0,1	Definition after publication
1	ID / 0xaa	0, 1, 0, 1	NOT / 0x55
2	NOT / 0x55	1, 0, 1, 0	ID / 0xaa
3	ID / 0xaa	0, 1, 0, 1	NOT / 0x55
4	NOT / 0x55	1, 0, 1, 0	ID / 0xaa

That exact alternating trace was verified across eight ticks. The example also quotes a bit from its own expression descriptor. Operators and descriptions are readable inputs, not only external labels.

What is modified, and when

Publications change declared custom truth coefficients **after** the complete tick. The current tick always sees one immutable definition snapshot. Each coefficient has one writer, with an explicitly selected source case. All cases share the resulting definition atlas. Per-case independent rule mutation is a different design and is not hidden here.

An actual capability, with a finite scope

The prototype can inspect and change its future **operator meaning**. It does not rewrite CUDA machine instructions, generate unlimited memory or prove general self-hosting. The example is an intentional oscillator, not a demonstration of convergence or biological self-replication.

This distinction is important: “self-referential” now names observable operations—quotation, delayed edges and controlled publication—rather than an analogy to a loop in a drawing.

What “low-level resident” can mean

Backend	Actual use	Tradeoff to measure
Global	Ordinary loads read the definition atlas and previous state.	Simple reference path; cache behavior remains hardware-managed.
Texture	Integer tex1Dfetch reads the atlas and prior state.	Uses unsigned element reads without filtering or float coordinates.
Shared	Each block loads the atlas into its allocated shared memory.	Explicit block-lifetime storage; staging and occupancy costs remain.

The kernel fuses expression evaluation into one calculation launch per tick and state shard. Each thread processes a word of 32 cases through the expression order. Neighboring threads access neighboring words of each field. There is no kernel launch for every individual expression.

Optional L2 priority

A requested **persisting L2 window** prioritizes the real definition atlas, capped to reported limits. NVIDIA describes preferential retention, not permanent residency. Shared memory, the texture path and L2 persistence are different mechanisms. The program reports applied sizes rather than pretending the cache is fully pinned. [E3; E4]

Warehouse versus workbench

VRAM is the warehouse. Cache is a small workbench. Putting more material in the warehouse does not enlarge the workbench. Filling cache with duplicate, unused definitions adds no computational meaning. The useful goal is to keep frequently reused definitions compact and accessible.

The optional CUDA L2 policy includes a float-typed host API field, `hitRatio=1.0f`. It is configuration metadata, not field arithmetic. The Boolean core contains no float types. Leaving the persistence option at zero omits that policy request entirely. [E4]

Current values are written to separate state memory and read by their owning thread through ordinary loads. Texture inputs remain unchanged during a launch. A later publication launch commits changed definitions. This avoids the incoherent same-launch texture read-after-write pattern documented by NVIDIA. [E3]

07 / R, THETA AND A ARE EXPLICIT STORAGE COORDINATES

Log-polar storage without trig

The state layout has three logical components: an expression row, an independent input case, and a temporal snapshot. The profile names them **R**, **Theta** and **A**. Their interpretation is declared once; the hot calculation follows integer references.

```
R: log_phi(r/r0) = expression_row / bins_per_phi
Theta: (case + 1/2) / case_count turns
A: T snapshot index; current and previous pages
word address: row * words_per_row + floor(case/32)
```

Each expression descriptor includes a Q16 radial log label. The base radius is $r_0 = 1$; the origin $r = 0$ is not included. Theta is an exact rational label. Physical radius metadata can be generated by integer square roots and fixed-point phi, but the Boolean evaluator does not consume it.

What was eliminated

No per-case sine, cosine, polar-to-Cartesian transform, floating-point distance or logarithm is calculated while a program runs. The connection graph already states which fields a node reads. This is the precise sense in which heavy coordinate math is bypassed for the selected language.

What was not established

A log-polar label does not by itself improve a cache hit rate. This prototype uses the layout for storage and reference, not as a geometric shortest-path solver. Angular cases do not exchange values with neighbors. There is no implicit fluid model, Klein seam, physical metric or apex equation.

Useful VRAM expansion

Extra VRAM holds more cases of the **same field program**. Variable-bit seeds can be generated directly from larger case indices on the GPU. Explicit patterns repeat, and cases repeat once the declared input variables run out of distinct combinations. The capacity test does not pretend repeated cases are new information.

The program can target a percentage of currently free VRAM. It subtracts known setup storage, uses two state pages and splits large resources at device limits. It writes back the base case set and reports all executed cases. Allocation overhead and concurrent GPU use can still prevent a requested near-full allocation.

08 / ACTUAL EXECUTED EVIDENCE

What passed the tests

Test family	Scope	Result
Primitive operator fields	256 tables × 8 scalar inputs	Passed
Packed primitive evaluation	32,768 additional random word cases	Passed
Complete executable programs	120 runs; 100 generated graphs	Matched scalar oracle
State comparison	276,224 resulting state bits	Matched
Adder	All 256 pairs of 4-bit inputs	Exact 5-bit sums
Counter	Eight tick counts; all supplied cases	Exact modulo-16 state
Self-definition	Eight inspected publication ticks	Correct rule/output alternation
Evaluator as fields	All 2,048 argument/coefficient combinations	Matched
Exact SDF codec	256 words; 12,800 integer queries	Passed
Sanitizers	Address + undefined behavior, CPU	Passed

These test families overlap. They are not added together into a misleading count of independent applications. The generated-program suite includes current references, delayed references, quotations and optional publication into custom definitions.

Independent verification

The fast C++ evaluator uses packed selections. The Python oracle reads each input bit separately and indexes a truth table. Tests compare the complete final state **and** definition atlas, not only a checksum or a few displayed outputs. Invalid unlocked cycles and malformed inputs are also checked.

The GPU boundary is explicit

The CPU implementation and these tests ran here. **CUDA compilation and GPU execution did not.** The environment has no nvcc and no NVIDIA GPU. CUDA source, build scripts and a three-backend GPU verification harness are included so the actual laptop can establish that result.

Evidence files: core_tests.json, integration_tests.json, core_sanitized.json, integration_sanitized.json and environment.json in the package's verification folder. The absence of a GPU result is recorded as NOT_RUN, not estimated from CPU speed.

09 / COMPARISONS ADDED AFTER THE IMPLEMENTATION FREEZE

Measured speed: gains and costs

Actual host: Intel Xeon Platinum 8370C; GCC 14.2.0; single-thread CPU execution. The benchmark uses 32,768 cases, six counter expressions and 64 ticks, with seven repetitions. Initial conversion, file I/O, final conversion and validation are outside the timer.

Representation of the same program	Median	Time / packed
Packed one-bit fields	3.726 ms	1.00
One uint8 Boolean per field/case	67.345 ms	18.07×
One FP32 Boolean per field/case	84.890 ms	22.78×

All three full final states agree. These are reference interpreters, not tuned production libraries. The FP32 baseline stores only Boolean zero/one values; it does not evaluate numerical SDF magnitudes. The result is not an RTX speed prediction.

A simpler native algorithm is faster

Same four counter output bits	Median
Packed field interpreter	3.782 ms
Specialized byte counter, modulo 16	0.503 ms

The specialized routine was **7.52× faster**. It computes the four output bits directly and does not materialize inspectable intermediate expressions or interpret changeable operator definitions. This is the cost of the general field-machine interface, not a contradiction of bit packing.

ELI5 conclusion

Packing lamps into strips can make a general lamp-program calculator much cheaper to run. But a tiny dedicated counter can still beat a general calculator. The correct question is which operation, representation and workload are being compared—not whether “binary” wins automatically.

The runtime/compiler hashes were frozen before comparative measurements and checked unchanged afterward. Raw samples and both comparison programs are included. GPU cache, temperature, power and throughput remain unmeasured here.

10 / EXACT STORAGE ARITHMETIC; NO COMPRESSION OF ARBITRARY DISTANCES

How much memory does it buy?

For one expression evaluated over **1,048,576 cases**, the bit-plane payload is straightforward:

Object	Packed Boolean fields	FP32 Boolean fields
One state snapshot	128 KiB	4 MiB
Previous + current snapshots	256 KiB	8 MiB
Two snapshots for 64 expressions	16 MiB	512 MiB

The data-plane ratio is exactly **32:1**, because both representations store the same Boolean information. The total allocation also includes definition snapshots, descriptors, publication records, input staging, driver resources and any requested output.

Actual supplied example sizes

Program	Expressions	Atlas	Two base state pages
Counter	6	512 B	384 B
Adder	16	832 B	1,024 B
Self-definition	11	832 B	704 B
Evaluator in fields	18	896 B	9,216 B
All ternary operators	259	8,608 B	16,576 B

These are logical payload sizes for the supplied case counts, not measured cache residency. A cache may store other working data and is shared with hardware activity. The atlas can be smaller than the cache without every definition remaining resident.

Where the remaining VRAM can be useful

Use it for more input assignments, more stateful scenarios or larger expression graphs supplied by the application. A GPU memory fill made of duplicate unused definitions is not automatically useful. The prototype’s budget option expands case state; it does not invent new semantics for unused bytes.

For a user-designed program with m independent input bits, only 2^m distinct input assignments exist. Allocating additional lanes beyond those combinations repeats work unless some other input or state parameter distinguishes them.

11 / DEMONSTRATED CAPABILITIES AND NATURAL EXTENSIONS

What this can be used for

Use	What this prototype contributes	Important boundary
Digital-logic checking	Evaluate many truth assignments; compare candidate Boolean circuits.	Not a complete hardware timing or electrical simulator.
Bit-sliced bounded arithmetic	Compose sum, carry, equality and selection fields.	Native arithmetic can be faster for a fixed simple task.
Boolean filtering / constraints	Return a packed acceptance field over supplied assignments.	Enumeration still needs the input combinations; no general NP shortcut.
Finite-state rule simulation	A/T delays and Boolean expressions evolve many scenarios.	No physical dynamics or sensor driver is supplied.
Adaptive rule experiments	Read and publish operator truth bits for subsequent ticks.	One shared definition atlas; control lanes are explicit.
Programmable binary pipelines	Change the field program without editing an opcode for every Boolean operation.	The fixed reader, memory addressing and snapshot rules remain.

An actual constraint example

The supplied predicate combines three clauses over four variables. It accepts exactly assignments **5, 6, 7 and 10** out of the sixteen possible assignments. Its output is a one-bit field. This is a useful packed calculation, without claiming the general search space disappeared.

What has not been turned into a working model

The package does not quietly substitute a claim of physical spacetime, universal self-replication, an optimal traveling-salesman solver or a continuous numerical SDF engine. The source's Klein-bottle and kinematic vocabulary remains a further modeling task. This release makes the **programs-as-fields** and **definitions-changing-definitions** parts executable and testable.

The strongest practical framing

A small, inspectable Boolean field virtual machine: its programs and operands share an SDF coding convention, it runs packed cases in parallel, and it supports clocked reflection. It is useful when that flexible representation is valuable—not merely because it can occupy VRAM.

12 / REPRODUCIBLE COMMANDS, WITH THE HARDWARE TEST FIRST

Build and run the laptop version

Your target is the RTX 5070 Ti Laptop with 12 GB as supplied. NVIDIA specifies 12 GB GDDR7 for that model, and CUDA 12.8 introduced SM120 compiler support. The build selects architecture 120; actual memory and cache limits are queried at runtime. [E5; E6]

Windows: Developer PowerShell

```
cmake -S . -B build -G "Visual Studio 17 2022" -A x64 `
-DAXISO_CUDA=ON -DAXISO_ARCH=120
cmake --build build --config Release --parallel
python tools/gpu_check.py --exe build/Release/axis0_gpu.exe `
--json verification/laptop_tests.json
```

Backticks continue commands onto the next line in PowerShell. The README also provides single-line versions. A CUDA-compatible C++ toolchain and CUDA's Visual Studio integration must be installed. CPU-only build instructions are supplied separately.

Run a self-changing definition

```
./build/Release/axis0_gpu.exe `
--program examples/self_definition.a0f `
--ticks 2 --defs texture --verify --out result.a0o `
python tools/readout.py result.a0o `
--manifest examples/self_definition.manifest.json
```

Measure cache and VRAM choices

```
python tools/gpu_metrics.py `
--exe build/Release/axis0_gpu.exe `
--output laptop_metrics.json
```

The script compares global, texture and shared definitions plus optional L2-priority windows. It records the case count, applied policy sizes, actual elapsed time and CPU verification. For large memory targets, free capacity may differ between runs; throughput is normalized by the work actually executed.

CUDA timing includes calculation launches, definition publication and completion. It excludes allocation, initialization, result copying and CPU verification. The tool reports no texture hit rate without a hardware profiler. Global and texture output equality is checked before interpreting speed differences.

13 / TRACEABLE CLAIMS AND IMPLEMENTATION ADDITIONS

Sources, boundaries and deliverables

[S1] Original six-page source beginning “men T room ...”: author-issued Axis 0 = ALL operators definition, page 5; initial vocabulary, pages 1–3.

[S2] Eight-page hyphenated-filename continuation: closure diagram, pages 4–5; entry/readout, pages 6–7; SDF = T, Present and Now, pages 7–8.

[S3] Six-page ladybug/Zaandam document: packed-bit proposal, page 1; Boolean double UU instruction, page 2; field/operator diagram and formula, page 3. Explanatory claims on later pages are not treated as measurements.

[E1] [Applications of Boolean Algebra: Claude Shannon and Circuit Design](#), the user-linked educational text. Supports the circuit/Boolean and truth-coefficient connection, not the claimed GPU behavior.

[E2] Claude E. Shannon, *A Symbolic Analysis of Relay and Switching Circuits*, *Electrical Engineering* 57(12), 713–723 (1938). DOI [10.1109/EE.1938.6431064](https://doi.org/10.1109/EE.1938.6431064).

[E3] [NVIDIA CUDA C++ Programming Guide 12.8.1](#), Sections 7.8.1.1 and 3.2.14.4: integer tex1Dfetch and read/write coherency.

[E4] [NVIDIA L2 Cache Control](#): device limits, persisting-access windows and preferential retention.

[E5] [NVIDIA CUDA 12.8 Features](#): SM120 compiler support. [E6] [NVIDIA RTX 50-series Laptop specifications](#). External pages consulted 18 September 2026.

What was introduced in this revision

The exact interval SDF codec, eight-bit operator catalogue, expression-field ABI, A/T delay rule, quotation and publication semantics, fused CUDA source, cache modes, examples and finite tests are implementation constructions. They are not retrospectively attributed to the source PDFs.

Inside the ZIP

C++ reference and CUDA evaluator; Python program compiler and readout; six example field programs; exact coding-field inspection; integer log-polar metadata tool; Windows/Linux build scripts; GPU correctness and metrics harnesses; complete model/ABI notes; CPU benchmarks, raw results, environment status and source hashes.

Final reading

This turns the goal into a **working finite Boolean field-program prototype**, with a tested CPU execution and an explicitly untested CUDA port. It removes heavy coordinate math from this calculation path by defining the representation and lowering its anchor tests to bits—not by asserting that all mathematical information or computational difficulty has vanished.